

管仲模拟器代码流程

config.py文件

代码块

```
1  import os
2  from enum import Enum
3
4  class Difficulty(Enum):
5      EASY = {"income_multiplier": 1.2, "stability_impact": 0.8}
6      NORMAL = {"income_multiplier": 1.0, "stability_impact": 1.0}
7      HARD = {"income_multiplier": 0.8, "stability_impact": 1.2}
8
9  class Config:
10     # 请替换为你自己的API密钥
11     DOUBAO_API_KEY = os.getenv("DOUBAO_API_KEY", "573ab249-3130-4926-be5f-da98998863ef")
12     DOUBAO_API_URL = os.getenv("DOUBAO_API_URL", "https://ark.cn-beijing.volces.com/api/v3/chat/completions")
13     ECONOMY_SETTINGS = {
14         "base_rates": {
15             "tax_income": 0.05,
16             "grain_yield": 0.15
17         },
18         "difficulty": Difficulty.NORMAL
19     }
20     HISTORICAL_EVENTS = {
21         "enable": True,
22         "frequency": 0.3 # 30%概率每年触发事件
23     }
```

economy.py文件

代码块

```
1  from flask_sqlalchemy import SQLAlchemy
2  from datetime import datetime
3
4  db = SQLAlchemy()
5
6  class EconomyModel(db.Model):
7      id = db.Column(db.Integer, primary_key=True)
```

```

8     name = db.Column(db.String(100), nullable=False)
9     description = db.Column(db.Text)
10    category = db.Column(db.String(50)) # 经济政策、思想等
11    created_at = db.Column(db.DateTime, default=datetime.utcnow)
12
13    def to_dict(self):
14        return {
15            'id': self.id,
16            'name': self.name,
17            'description': self.description,
18            'category': self.category,
19            'created_at': self.created_at.isoformat()
20        }

```

user.py文件

代码块

```

1  from flask_sqlalchemy import SQLAlchemy
2  from datetime import datetime
3
4  db = SQLAlchemy()
5
6  class User(db.Model):
7      id = db.Column(db.Integer, primary_key=True)
8      username = db.Column(db.String(80), unique=True, nullable=False)
9      password = db.Column(db.String(120), nullable=False)
10     created_at = db.Column(db.DateTime, default=datetime.utcnow)
11
12     def to_dict(self):
13         return {
14             'id': self.id,
15             'username': self.username,
16             'created_at': self.created_at.isoformat()
17         }

```

auth.py文件

代码块

```

1  from flask import Blueprint, request, jsonify
2  from flask_jwt_extended import create_access_token
3  from backend.models.user import User, db
4
5  auth_bp = Blueprint('auth', __name__)
6

```

```
7 @auth_bp.route('/api/auth/register', methods=['POST'])
8 def register():
9     """用户注册"""
10    data = request.get_json()
11    username = data.get('username')
12    password = data.get('password')
13
14    if not username or not password:
15        return jsonify({"message": "用户名和密码不能为空"}), 400
16
17    if User.query.filter_by(username=username).first():
18        return jsonify({"message": "用户名已存在"}), 400
19
20    user = User(username=username, password=password)
21    db.session.add(user)
22    db.session.commit()
23
24    return jsonify({"message": "注册成功"}), 201
25
26 @auth_bp.route('/api/auth/login', methods=['POST'])
27 def login():
28     """用户登录"""
29    data = request.get_json()
30    username = data.get('username')
31    password = data.get('password')
32
33    user = User.query.filter_by(username=username, password=password).first()
34
35    if user:
36        access_token = create_access_token(identity=user.id)
37        return jsonify({
38            "access_token": access_token,
39            "user_id": user.id,
40            "username": user.username
41        }), 200
42    else:
43        return jsonify({"message": "用户名或密码错误"}), 401
44
45 @auth_bp.route('/api/auth/user', methods=['GET'])
46 def get_user_info():
47     """获取用户信息（示例）"""
48    return jsonify({
49        "message": "用户信息接口",
50        "endpoint": "/api/auth/user"
51    })
```

dialogue.py文件

代码块

```
1  from flask import Blueprint, request, jsonify
2  from flask_jwt_extended import jwt_required, get_jwt_identity
3  from backend.models.user import db
4
5  dialogue_bp = Blueprint('dialogue', __name__)
6
7  # 临时对话存储 (实际项目中使用数据库)
8  dialogues = {}
9
10 @dialogue_bp.route('/api/dialogues', methods=['POST'])
11 @jwt_required()
12 def create_dialogue():
13     """创建新对话"""
14     data = request.get_json() or {}
15     user_id = get_jwt_identity()
16
17     dialogue_id = len(dialogues) + 1
18     dialogues[dialogue_id] = {
19         'id': dialogue_id,
20         'user_id': user_id,
21         'title': data.get('title', '新对话'),
22         'messages': [],
23         'created_at': '2025-01-20T10:00:00'
24     }
25
26     return jsonify(dialogues[dialogue_id]), 201
27
28 @dialogue_bp.route('/api/dialogues', methods=['GET'])
29 @jwt_required()
30 def get_dialogues():
31     """获取对话列表"""
32     user_id = get_jwt_identity()
33     user_dialogues = [d for d in dialogues.values() if d['user_id'] == user_id]
34
35     return jsonify({
36         'dialogues': user_dialogues,
37         'total': len(user_dialogues)
38     }), 200
39
40 @dialogue_bp.route('/api/dialogues/<int:dialogue_id>', methods=['GET'])
41 @jwt_required()
42 def get_dialogue(dialogue_id):
43     """获取对话详情"""
```

```
44     dialogue = dialogues.get(dialogue_id)
45     if not dialogue:
46         return jsonify({"message": "对话不存在"}), 404
47
48     user_id = get_jwt_identity()
49     if dialogue['user_id'] != user_id:
50         return jsonify({"message": "无权访问此对话"}), 403
51
52     return jsonify(dialogue), 200
53
54 @dialogue_bp.route('/api/dialogues/<int:dialogue_id>/messages', methods=
55 ['POST'])
56 @jwt_required()
57 def add_message(dialogue_id):
58     """添加消息到对话"""
59     data = request.get_json()
60     message_content = data.get('message')
61
62     if not message_content:
63         return jsonify({"message": "消息内容不能为空"}), 400
64
65     dialogue = dialogues.get(dialogue_id)
66     if not dialogue:
67         return jsonify({"message": "对话不存在"}), 404
68
69     user_id = get_jwt_identity()
70     if dialogue['user_id'] != user_id:
71         return jsonify({"message": "无权访问此对话"}), 403
72
73     # 添加用户消息
74     dialogue['messages'].append({
75         'sender': 'user',
76         'content': message_content,
77         'timestamp': '2025-01-20T10:00:00'
78     })
79
80     # 模拟AI回复
81     dialogue['messages'].append({
82         'sender': 'guanzhong',
83         'content': f"收到您的消息：{message_content}。管仲正在思考中...",
84         'timestamp': '2025-01-20T10:01:00'
85     })
86
87     return jsonify({
88         'message': '消息添加成功',
89         'dialogue_id': dialogue_id
90     }), 200
```

economy.py文件

代码块

```
1  from flask_sqlalchemy import SQLAlchemy
2  from datetime import datetime
3
4  db = SQLAlchemy()
5
6  class EconomyModel(db.Model):
7      id = db.Column(db.Integer, primary_key=True)
8      name = db.Column(db.String(100), nullable=False)
9      description = db.Column(db.Text)
10     category = db.Column(db.String(50)) # 经济政策、思想等
11     created_at = db.Column(db.DateTime, default=datetime.utcnow)
12
13     def to_dict(self):
14         return {
15             'id': self.id,
16             'name': self.name,
17             'description': self.description,
18             'category': self.category,
19             'created_at': self.created_at.isoformat()
20         }
```

user.py文件

代码块

```
1  from flask_sqlalchemy import SQLAlchemy
2  from datetime import datetime
3
4  db = SQLAlchemy()
5
6  class User(db.Model):
7      id = db.Column(db.Integer, primary_key=True)
8      username = db.Column(db.String(80), unique=True, nullable=False)
9      password = db.Column(db.String(120), nullable=False)
10     created_at = db.Column(db.DateTime, default=datetime.utcnow)
11
12     def to_dict(self):
13         return {
14             'id': self.id,
15             'username': self.username,
16             'created_at': self.created_at.isoformat()
17         }
```

auth.py文件

代码块

```
1  from flask import Blueprint, request, jsonify
2  from flask_jwt_extended import create_access_token
3  from backend.models.user import User, db
4
5  auth_bp = Blueprint('auth', __name__)
6
7  @auth_bp.route('/api/auth/register', methods=['POST'])
8  def register():
9      """用户注册"""
10     data = request.get_json()
11     username = data.get('username')
12     password = data.get('password')
13
14     if not username or not password:
15         return jsonify({"message": "用户名和密码不能为空"}), 400
16
17     if User.query.filter_by(username=username).first():
18         return jsonify({"message": "用户名已存在"}), 400
19
20     user = User(username=username, password=password)
21     db.session.add(user)
22     db.session.commit()
23
24     return jsonify({"message": "注册成功"}), 201
25
26  @auth_bp.route('/api/auth/login', methods=['POST'])
27  def login():
28      """用户登录"""
29     data = request.get_json()
30     username = data.get('username')
31     password = data.get('password')
32
33     user = User.query.filter_by(username=username, password=password).first()
34
35     if user:
36         access_token = create_access_token(identity=user.id)
37         return jsonify({
38             "access_token": access_token,
39             "user_id": user.id,
40             "username": user.username
```

```

41         }), 200
42     else:
43         return jsonify({"message": "用户名或密码错误"}), 401
44
45 @auth_bp.route('/api/auth/user', methods=['GET'])
46 def get_user_info():
47     """获取用户信息（示例）"""
48     return jsonify({
49         "message": "用户信息接口",
50         "endpoint": "/api/auth/user"
51     })

```

dialogue.py

代码块

```

1  from flask import Blueprint, request, jsonify
2  from flask_jwt_extended import jwt_required, get_jwt_identity
3  from backend.models.user import db
4
5  dialogue_bp = Blueprint('dialogue', __name__)
6
7  # 临时对话存储（实际项目中使用数据库）
8  dialogues = {}
9
10 @dialogue_bp.route('/api/dialogues', methods=['POST'])
11 @jwt_required()
12 def create_dialogue():
13     """创建新对话"""
14     data = request.get_json() or {}
15     user_id = get_jwt_identity()
16
17     dialogue_id = len(dialogues) + 1
18     dialogues[dialogue_id] = {
19         'id': dialogue_id,
20         'user_id': user_id,
21         'title': data.get('title', '新对话'),
22         'messages': [],
23         'created_at': '2025-01-20T10:00:00'
24     }
25
26     return jsonify(dialogues[dialogue_id]), 201
27
28 @dialogue_bp.route('/api/dialogues', methods=['GET'])
29 @jwt_required()
30 def get_dialogues():

```



```
31     """获取对话列表"""
32     user_id = get_jwt_identity()
33     user_dialogues = [d for d in dialogues.values() if d['user_id'] == user_id]
34
35     return jsonify({
36         'dialogues': user_dialogues,
37         'total': len(user_dialogues)
38     }), 200
39
40 @dialogue_bp.route('/api/dialogues/<int:dialogue_id>', methods=['GET'])
41 @jwt_required()
42 def get_dialogue(dialogue_id):
43     """获取对话详情"""
44     dialogue = dialogues.get(dialogue_id)
45     if not dialogue:
46         return jsonify({"message": "对话不存在"}), 404
47
48     user_id = get_jwt_identity()
49     if dialogue['user_id'] != user_id:
50         return jsonify({"message": "无权访问此对话"}), 403
51
52     return jsonify(dialogue), 200
53
54 @dialogue_bp.route('/api/dialogues/<int:dialogue_id>/messages', methods=
55 ['POST'])
56 @jwt_required()
57 def add_message(dialogue_id):
58     """添加消息到对话"""
59     data = request.get_json()
60     message_content = data.get('message')
61
62     if not message_content:
63         return jsonify({"message": "消息内容不能为空"}), 400
64
65     dialogue = dialogues.get(dialogue_id)
66     if not dialogue:
67         return jsonify({"message": "对话不存在"}), 404
68
69     user_id = get_jwt_identity()
70     if dialogue['user_id'] != user_id:
71         return jsonify({"message": "无权访问此对话"}), 403
72
73     # 添加用户消息
74     dialogue['messages'].append({
75         'sender': 'user',
76         'content': message_content,
77         'timestamp': '2025-01-20T10:00:00'
```

```

77     })
78
79     # 模拟AI回复
80     dialogue['messages'].append({
81         'sender': 'guanzhong',
82         'content': f"收到您的消息: {message_content}。管仲正在思考中...",
83         'timestamp': '2025-01-20T10:01:00'
84     })
85
86     return jsonify({
87         'message': '消息添加成功',
88         'dialogue_id': dialogue_id
89     }), 200

```

economy.py

代码块

```

1  from flask import Blueprint, request, jsonify
2  from backend.models.economy import EconomyModel, db
3
4  economy_bp = Blueprint('economy', __name__)
5
6  @economy_bp.route('/api/economy/concepts', methods=['GET'])
7  def get_economy_concepts():
8      """获取管仲经济思想概念"""
9      concepts = [
10         {
11             "id": 1,
12             "name": "相地而衰征",
13             "category": "土地政策",
14             "description": "根据土地优劣征收不同赋税的土地改革政策",
15             "importance": 95
16         },
17         {
18             "id": 2,
19             "name": "轻重九府",
20             "category": "经济调控",
21             "description": "国家调控市场、平衡物价的经济管理方法",
22             "importance": 90
23         },
24         {
25             "id": 3,
26             "name": "官山海",
27             "category": "资源管理",
28             "description": "国家垄断盐铁等重要资源的管理政策",

```

```
29         "importance": 85
30     }
31 ]
32 return jsonify(concepts), 200
33
34 @economy_bp.route('/api/economy/graph', methods=['GET'])
35 def get_economy_graph():
36     """获取经济思想知识图谱"""
37     graph_data = {
38         "nodes": [
39             {"id": "gz", "label": "管仲", "category": "人物", "size": 30},
40             {"id": "jj", "label": "经济思想", "category": "思想", "size": 25},
41             {"id": "xd", "label": "相地而衰征", "category": "政策", "size": 20},
42             {"id": "qz", "label": "轻重九府", "category": "政策", "size": 20},
43             {"id": "gsh", "label": "官山海", "category": "政策", "size": 20}
44         ],
45         "edges": [
46             {"source": "gz", "target": "jj", "label": "提出"},
47             {"source": "jj", "target": "xd", "label": "包含"},
48             {"source": "jj", "target": "qz", "label": "包含"},
49             {"source": "jj", "target": "gsh", "label": "包含"}
50         ]
51     }
52     return jsonify(graph_data), 200
53
54 @economy_bp.route('/api/economy/simulate', methods=['POST'])
55 def simulate_economy():
56     """经济政策模拟"""
57     data = request.get_json()
58     policy = data.get('policy')
59     parameters = data.get('parameters', {})
60
61     # 模拟不同政策的效果
62     simulations = {
63         "相地而衰征": {
64             "effect": "财政收入增加20%",
65             "stability": "社会稳定性提高",
66             "duration": "长期有效"
67         },
68         "轻重九府": {
69             "effect": "市场物价稳定",
70             "stability": "经济波动减少",
71             "duration": "中期调控"
72         },
73         "官山海": {
74             "effect": "国家资源收入大幅增加",
75             "stability": "资源分配更合理",
```

```

76         "duration": "长期政策"
77     }
78 }
79
80 result = simulations.get(policy, {
81     "effect": "政策效果待评估",
82     "stability": "未知",
83     "duration": "需要更多数据"
84 })
85
86 return jsonify({
87     "policy": policy,
88     "parameters": parameters,
89     "simulation_result": result
90 }, 200

```

game.py

代码块

```

1  from flask import Blueprint, request, jsonify
2
3  game_bp = Blueprint('game', __name__)
4
5  @game_bp.route('/api/game/scenarios', methods=['GET'])
6  def get_game_scenarios():
7      """获取历史场景列表"""
8      scenarios = [
9          {
10             "id": 1,
11             "title": "九合诸侯",
12             "description": "体验管仲辅佐齐桓公'九合诸侯，一匡天下'的历史场景",
13             "difficulty": "中等",
14             "estimated_time": 45,
15             "image": "/static/images/scenario1.jpg"
16         },
17         {
18             "id": 2,
19             "title": "相地而衰征",
20             "description": "模拟管仲实施土地改革政策的历史场景",
21             "difficulty": "简单",
22             "estimated_time": 30,
23             "image": "/static/images/scenario2.jpg"
24         },
25         {
26             "id": 3,

```

```
27         "title": "管鲍之交",
28         "description": "重现管仲与鲍叔牙深厚友谊的历史场景",
29         "difficulty": "简单",
30         "estimated_time": 25,
31         "image": "/static/images/scenario3.jpg"
32     }
33 ]
34 return jsonify(scenarios), 200
35
36 @game_bp.route('/api/game/scenarios/<int:scenario_id>', methods=['GET'])
37 def get_scenario_detail(scenario_id):
38     """获取场景详情"""
39     scenarios_data = {
40         1: {
41             "id": 1,
42             "title": "九合诸侯",
43             "description": "详细描述...",
44             "objectives": ["达成外交协议", "维护齐国霸权"],
45             "characters": ["管仲", "齐桓公", "诸侯"],
46             "duration": "45分钟"
47         },
48         2: {
49             "id": 2,
50             "title": "相地而衰征",
51             "description": "详细描述...",
52             "objectives": ["实施土地改革", "平衡赋税"],
53             "characters": ["管仲", "土地官员", "农民"],
54             "duration": "30分钟"
55         },
56         3: {
57             "id": 3,
58             "title": "管鲍之交",
59             "description": "详细描述...",
60             "objectives": ["建立信任关系", "展现才能"],
61             "characters": ["管仲", "鲍叔牙", "齐桓公"],
62             "duration": "25分钟"
63         }
64     }
65
66     scenario = scenarios_data.get(scenario_id)
67     if not scenario:
68         return jsonify({"message": "场景不存在"}), 404
69
70     return jsonify(scenario), 200
71
72 @game_bp.route('/api/game/start/<int:scenario_id>', methods=['POST'])
73 def start_scenario(scenario_id):
```

```

74     """开始游戏场景"""
75     return jsonify({
76         "scenario_id": scenario_id,
77         "status": "started",
78         "message": "场景已开始，请按照指引进行操作",
79         "first_step": "了解当前局势和任务目标"
80     }), 200
81
82 @game_bp.route('/api/game/action', methods=['POST'])
83 def game_action():
84     """执行游戏动作"""
85     data = request.get_json()
86     action = data.get('action')
87     scenario_id = data.get('scenario_id')
88
89     # 模拟动作结果
90     results = {
91         "diplomacy": "外交成功，诸侯信服",
92         "economic_reform": "经济改革初见成效",
93         "military": "军事部署完成",
94         "advice": "建议被采纳"
95     }
96
97     result = results.get(action, "动作执行中...")
98
99     return jsonify({
100         "action": action,
101         "result": result,
102         "next_step": "请继续下一步操作",
103         "score_change": +10
104     }), 200

```

api_design.py

代码块

```

1  # 管仲AI后端API设计规范
2  # 版本: v1.0
3  # 最后更新: 2025-01-20
4
5  """
6  管仲AI后端API设计文档
7  基于OpenAPI 3.0规范，与实际代码保持同步
8  """
9
10 # =====

```

```
11 # 1. 用户认证模块
12 # =====
13
14 """
15 POST /api/auth/register
16 描述：用户注册
17 请求体：
18 {
19     "username": "string, 必需",
20     "password": "string, 必需"
21 }
22 响应：
23 - 201: {"message": "注册成功"}
24 - 400: {"message": "用户名已存在"}
25 """
26
27 """
28 POST /api/auth/login
29 描述：用户登录
30 请求体：
31 {
32     "username": "string, 必需",
33     "password": "string, 必需"
34 }
35 响应：
36 - 200: {"access_token": "JWT令牌"}
37 - 401: {"message": "认证失败"}
38 """
39
40 """
41 POST /api/auth/logout
42 描述：用户登出（客户端清除token）
43 认证：需要Bearer Token
44 响应：
45 - 200: {"message": "登出成功"}
46 """
47
48 """
49 GET /api/auth/user
50 描述：获取当前用户信息
51 认证：需要Bearer Token
52 响应：
53 - 200: {"id": 1, "username": "user123", "created_at": "2025-01-20T10:00:00"}
54 - 401: {"message": "未认证"}
55 """
56
57 # =====
```

```
58 # 2. 对话管理模块
59 # =====
60
61 """
62 POST /api/dialogues
63 描述：创建新对话
64 认证：需要Bearer Token
65 请求体：
66 {
67     "title": "string, 可选，默认'新对话'",
68     "tags": ["string", "可选标签"]
69 }
70 响应：
71 - 201: {"id": 1, "title": "对话标题", "created_at": "2025-01-20T10:00:00"}
72 """
73
74 """
75 GET /api/dialogues
76 描述：获取对话列表（分页）
77 认证：需要Bearer Token
78 查询参数：
79 - page: integer, 页码，默认1
80 - size: integer, 每页大小，默认10
81 响应：
82 - 200: {
83     "dialogues": [
84         {"id": 1, "title": "标题", "created_at": "2025-01-20T10:00:00",
85         "message_count": 5}
86     ],
87     "total": 100,
88     "page": 1,
89     "size": 10
90 }
91 """
92
93 GET /api/dialogues/{id}
94 描述：获取对话详情
95 认证：需要Bearer Token
96 路径参数：
97 - id: integer, 对话ID
98 响应：
99 - 200: {
100     "id": 1,
101     "title": "标题",
102     "created_at": "2025-01-20T10:00:00",
103     "messages": [
```



```
104         {"id": 1, "sender": "user", "content": "内容", "created_at": "2025-01-
105         ]
106     }
107     - 404: {"message": "对话不存在"}
108     ""
109
110     ""
111     GET /api/dialogues/{id}/messages
112     描述: 获取对话消息 (分页)
113     认证: 需要Bearer Token
114     查询参数:
115     - page: integer, 页码, 默认1
116     - size: integer, 每页大小, 默认20
117     响应:
118     - 200: {
119         "messages": [消息列表],
120         "total": 50,
121         "page": 1,
122         "size": 20
123     }
124     ""
125
126     # =====
127     # 3. 管仲AI核心模块
128     # =====
129
130     ""
131     POST /api/guanzhong/chat
132     描述: 与管仲AI进行实时对话
133     认证: 可选 (游客模式支持)
134     请求体:
135     {
136         "message": "string, 必需, 用户消息",
137         "dialogue_id": "integer, 可选, 关联对话ID"
138     }
139     响应:
140     - 200: {
141         "ai_reply": "管仲AI的回复",
142         "dialogue_id": "当前对话ID",
143         "thought_process": {"economic": true, "political": false}
144     }
145     - 400: {"message": "请提供消息内容"}
146     - 500: {"message": "AI服务暂时不可用"}
147     ""
148
149     ""
```

```
150 POST /api/guanzhong/think
151 描述：管仲AI深度思考（异步处理复杂问题）
152 认证：需要Bearer Token
153 请求体：
154 {
155     "question": "string, 必需, 复杂问题",
156     "background": "string, 可选, 背景信息",
157     "thinking_time": "integer, 可选, 思考时间（秒）"
158 }
159 响应：
160 - 202: {"task_id": "uuid", "status": "processing", "estimated_time": 30}
161 ""
162
163 ""
164 GET /api/guanzhong/topics
165 描述：获取管仲思想主题列表
166 响应：
167 - 200: [
168     {"id": 1, "title": "经济思想", "description": "仓廩实而知礼节...", "icon":
169     "💰"}
170 ]
171 ""
172 ""
173 GET /api/guanzhong/topics/{id}
174 描述：获取主题详情
175 路径参数：
176 - id: integer, 主题ID
177 响应：
178 - 200: {
179     "id": 1,
180     "title": "经济思想",
181     "description": "详细描述...",
182     "key_concepts": ["概念1", "概念2"],
183     "related_topics": [2, 3]
184 }
185 - 404: {"message": "主题不存在"}
186 ""
187
188 # =====
189 # 4. 知识图谱模块
190 # =====
191
192 ""
193 GET /api/knowledge/graph
194 描述：获取管仲思想知识图谱
195 查询参数：
```

```
196 - depth: integer, 图谱深度, 默认2
197 - root: string, 根节点, 默认"管仲"
198 响应:
199 - 200: {
200     "nodes": [
201         {"id": "gz", "label": "管仲", "category": "人物", "size": 30}
202     ],
203     "edges": [
204         {"source": "gz", "target": "jj", "label": "提出"}
205     ]
206 }
207 ""
208
209 ""
210 GET /api/knowledge/concepts
211 描述: 获取概念列表 (支持搜索)
212 查询参数:
213 - q: string, 搜索关键词
214 - category: string, 分类过滤
215 - page: integer, 页码
216 响应:
217 - 200: {
218     "concepts": [
219         {"id": 1, "name": "相地而衰征", "category": "经济政策", "description":
220         "..."}
221     ],
222     "total": 100
223 }
224 ""
225 ""
226 GET /api/knowledge/concepts/{id}
227 描述: 获取概念详情
228 响应:
229 - 200: 概念详细信息
230 - 404: {"message": "概念不存在"}
231 ""
232
233 ""
234 GET /api/knowledge/relations
235 描述: 获取关系列表
236 查询参数:
237 - source: string, 源概念
238 - target: string, 目标概念
239 - type: string, 关系类型
240 响应:
241 - 200: 关系列表
```

```
242  """
243
244  # =====
245  # 5. 历史场景模块
246  # =====
247
248  """
249  GET /api/scenarios
250  描述：获取历史场景列表
251  响应：
252  - 200: [
253      {
254          "id": 1,
255          "title": "九合诸侯",
256          "description": "场景描述...",
257          "image_url": "图片URL",
258          "difficulty": "easy|medium|hard",
259          "estimated_time": 30
260      }
261  ]
262  """
263
264  """
265  GET /api/scenarios/{id}
266  描述：获取场景详情
267  响应：
268  - 200: 场景详细信息
269  - 404: {"message": "场景不存在"}
270  """
271
272  """
273  POST /api/scenarios/{id}/enter
274  描述：进入历史场景
275  认证：需要Bearer Token
276  响应：
277  - 200: {"session_id": "场景会话ID", "instructions": "场景说明"}
278  """
279
280  """
281  POST /api/scenarios/{id}/action
282  描述：在场景中执行动作
283  认证：需要Bearer Token
284  请求体：
285  {
286      "session_id": "string, 必需, 场景会话ID",
287      "action": "string, 必需, 执行的动作",
288      "parameters": "object, 可选, 动作参数"
```

```

289 }
290 响应:
291 - 200: {"result": "动作结果", "next_instructions": "下一步说明"}
292 ""
293
294 # =====
295 # 6. 系统健康检查
296 # =====
297
298 ""
299 GET /api/health
300 描述: 系统健康检查
301 响应:
302 - 200: {
303     "status": "healthy",
304     "timestamp": "2025-01-20T10:00:00",
305     "version": "1.0.0",
306     "services": {
307         "database": "connected",
308         "ai_service": "available"
309     }
310 }
311 ""
312
313 ""
314 GET /api/version
315 描述: 获取API版本信息
316 响应:
317 - 200: {
318     "name": "管仲AI后端API",
319     "version": "1.0.0",
320     "description": "管仲思想AI对话平台"
321 }
322 ""

```

app_backup.py

代码块

```

1  # backend/app.py
2  from flask import Flask
3  from flask_sqlalchemy import SQLAlchemy
4  from backend.models.user import db
5
6  app = Flask(__name__)
7  app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///game.db'

```

```

8  app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
9  db.init_app(app)
10
11  with app.app_context():
12      db.create_all()
13
14  # 导入路由
15  from backend.routes.dialogue import bp as dialogue_bp
16  from backend.routes.game import bp as game_bp
17  from backend.routes.economy import bp as economy_bp
18
19  app.register_blueprint(dialogue_bp)
20  app.register_blueprint(game_bp)
21  app.register_blueprint(economy_bp)
22
23  if __name__ == '__main__':
24      app.run(debug=True)

```

app.py

代码块

```

1  # 新增导入
2  import requests
3  import json
4  import os
5  import logging
6  from threading import Lock
7  from datetime import datetime
8  from flask import Flask, request, jsonify, send_from_directory
9  from flask_cors import CORS
10
11  # 正确初始化Flask应用
12  app = Flask(__name__)
13  CORS(app) # 允许跨域请求
14
15  # 配置日志
16  logging.basicConfig(level=logging.INFO)
17  logger = logging.getLogger(__name__)
18
19  # 豆包API配置 (使用您提供的API Key)
20  app.config['DOUBAO_API_KEY'] = '573ab249-3130-4926-be5f-da98998863ef'
21  app.config['DOUBAO_API_URL'] = 'https://ark.cn-
    beijing.volces.com/api/v3/chat/completions'
22
23  # 线程安全锁

```

```
24 dialogue_lock = Lock()
25
26 class DoubaoAIClient:
27     @staticmethod
28     def generate_reply(user_message):
29         """调用豆包大模型生成管仲风格回复"""
30         system_prompt = """
31         你正在扮演春秋时期齐国名相管仲。请严格遵循以下角色设定：
32
33         角色背景：
34         - 春秋时期齐国名相，辅佐齐桓公成为春秋五霸之首
35         - 思想核心：富民强国、礼义廉耻、尊王攘夷
36         - 著名主张："仓廩实而知礼节，衣食足而知荣辱"
37         - 政策：相地而衰征、官山海、轻重九府、四民分业
38
39         回答要求：
40         1. 使用文言文与现代汉语结合的风格（70%文言+30%现代）
41         2. 体现管仲的治国智慧和哲学思想
42         3. 适当引用《管子》中的经典语句
43         4. 保持谦逊但权威的语气
44         5. 回答要精炼，控制在100-200字
45         6. 以"善哉！"、"吾闻之"等春秋时期用语开头
46
47         示例回答：
48         "善哉！治国之道，首在富民。仓廩实而知礼节，衣食足而知荣辱。民富则易治，民贫则难
49         安。"
50
51         """
52
53     try:
54         with dialogue_lock:
55             headers = {
56                 'Authorization': f'Bearer {app.config["DOUBAO_API_KEY"]}',
57                 'Content-Type': 'application/json'
58             }
59
60             payload = {
61                 "model": "doubao-seed-1-6-250615", # 根据实际模型调整
62                 "messages": [
63                     {"role": "system", "content": system_prompt},
64                     {"role": "user", "content": user_message}
65                 ],
66                 "temperature": 0.7,
67                 "max_tokens": 500,
68                 "stream": False
69             }
```

```

69         logger.info(f"发送请求到豆包API: {payload['messages'][1]
['content'][:50]}...")
70
71         response = requests.post(
72             app.config['DOUBAO_API_URL'],
73             headers=headers,
74             json=payload,
75             timeout=30
76         )
77
78         if response.status_code != 200:
79             error_msg = f"API请求失败: {response.status_code} -
{response.text}"
80             logger.error(error_msg)
81             raise Exception(error_msg)
82
83             result = response.json()
84             logger.info("豆包API响应成功")
85             return result['choices'][0]['message']['content']
86
87     except Exception as e:
88         logger.error(f"豆包API调用失败: {e}")
89         return f"吾思有所阻, 请稍后再试。错误: {str(e)}"
90
91 # 豆包对话API路由
92 @app.route('/api/doubao/chat', methods=['POST'])
93 def doubao_chat():
94     """豆包大模型对话接口"""
95     try:
96         data = request.get_json()
97         user_message = data.get('message', '').strip()
98
99         if not user_message:
100             return jsonify({"error": "消息内容不能为空"}), 400
101
102         logger.info(f"收到用户消息: {user_message}")
103
104         # 调用豆包API
105         ai_reply = DoubaoAIClient.generate_reply(user_message)
106
107         return jsonify({
108             "success": True,
109             "ai_reply": ai_reply,
110             "model": "doubao",
111             "api_key": app.config['DOUBAO_API_KEY'][:8] + "... " +
app.config['DOUBAO_API_KEY'][-4:],
112             "timestamp": datetime.now().isoformat()

```



```
113         })
114
115     except Exception as e:
116         logger.error(f"对话处理错误: {e}")
117         return jsonify({
118             "success": False,
119             "error": str(e),
120             "ai_reply": "管仲沉思中, 请稍候再试"
121         }), 500
122
123     # 豆包API健康检查
124     @app.route('/api/doubao/health', methods=['GET'])
125     def doubao_health():
126         """检查豆包API状态"""
127         try:
128             # 简单的测试请求
129             test_reply = DoubaoAIClient.generate_reply("你好")
130             return jsonify({
131                 "status": "healthy",
132                 "model": "doubao",
133                 "api_key_status": "有效",
134                 "response_time": "正常",
135                 "timestamp": datetime.now().isoformat()
136             })
137         except Exception as e:
138             return jsonify({
139                 "status": "unhealthy",
140                 "error": str(e),
141                 "api_key_status": "无效",
142                 "timestamp": datetime.now().isoformat()
143             }), 503
144
145     # 静态文件服务
146     @app.route('/')
147     def serve_index():
148         """提供前端主页面"""
149         return send_from_directory('../frontend/public', 'guanzhong_ai.html')
150
151     @app.route('/<path:path>')
152     def serve_static(path):
153         """提供静态文件"""
154         return send_from_directory('../frontend/public', path)
155
156     # 应用启动
157     if __name__ == '__main__':
158         # 检查前端目录是否存在
159         frontend_path = os.path.abspath('../frontend/public')
```

```

160     if not os.path.exists(frontend_path):
161         logger.warning(f"前端目录不存在: {frontend_path}")
162
163     # 启动Flask应用
164     app.run(
165         debug=True,
166         port=5000,
167         host='0.0.0.0',
168         threaded=True
169     )

```

index.html

代码块

```

1  # 新增导入
2  import requests
3  import json
4  import os
5  import logging
6  from threading import Lock
7  from datetime import datetime
8  from flask import Flask, request, jsonify, send_from_directory
9  from flask_cors import CORS
10
11 # 正确初始化Flask应用
12 app = Flask(__name__)
13 CORS(app) # 允许跨域请求
14
15 # 配置日志
16 logging.basicConfig(level=logging.INFO)
17 logger = logging.getLogger(__name__)
18
19 # 豆包API配置 (使用您提供的API Key)
20 app.config['DOUBAO_API_KEY'] = '573ab249-3130-4926-be5f-da98998863ef'
21 app.config['DOUBAO_API_URL'] = 'https://ark.cn-
    beijing.volces.com/api/v3/chat/completions'
22
23 # 线程安全锁
24 dialogue_lock = Lock()
25
26 class DoubaoAIClient:
27     @staticmethod
28     def generate_reply(user_message):
29         """调用豆包大模型生成管仲风格回复"""
30         system_prompt = """

```

你正在扮演春秋时期齐国名相管仲。请严格遵循以下角色设定：

角色背景：

- 春秋时期齐国名相，辅佐齐桓公成为春秋五霸之首
- 思想核心：富民强国、礼义廉耻、尊王攘夷
- 著名主张："仓廩实而知礼节，衣食足而知荣辱"
- 政策：相地而衰征、官山海、轻重九府、四民分业

回答要求：

1. 使用文言文与现代汉语结合的风格（70%文言+30%现代）
2. 体现管仲的治国智慧和哲学思想
3. 适当引用《管子》中的经典语句
4. 保持谦逊但权威的语气
5. 回答要精炼，控制在100-200字
6. 以"善哉！"、"吾闻之"等春秋时期用语开头

示例回答：

"善哉！治国之道，首在富民。仓廩实而知礼节，衣食足而知荣辱。民富则易治，民贫则难安。"

```
"""

try:
    with dialogue_lock:
        headers = {
            'Authorization': f'Bearer {app.config["DOUBAO_API_KEY"]}',
            'Content-Type': 'application/json'
        }

        payload = {
            "model": "doubao-seed-1-6-250615", # 根据实际模型调整
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_message}
            ],
            "temperature": 0.7,
            "max_tokens": 500,
            "stream": False
        }

        logger.info(f"发送请求到豆包API: {payload['messages'][1]
['content'][:50]}...")

        response = requests.post(
            app.config['DOUBAO_API_URL'],
            headers=headers,
            json=payload,
            timeout=30
```

```

76         )
77
78         if response.status_code != 200:
79             error_msg = f"API请求失败: {response.status_code} -
{response.text}"
80             logger.error(error_msg)
81             raise Exception(error_msg)
82
83             result = response.json()
84             logger.info("豆包API响应成功")
85             return result['choices'][0]['message']['content']
86
87     except Exception as e:
88         logger.error(f"豆包API调用失败: {e}")
89         return f"吾思有所阻, 请稍后再试。错误: {str(e)}"
90
91 # 豆包对话API路由
92 @app.route('/api/doubao/chat', methods=['POST'])
93 def doubao_chat():
94     """豆包大模型对话接口"""
95     try:
96         data = request.get_json()
97         user_message = data.get('message', '').strip()
98
99         if not user_message:
100             return jsonify({"error": "消息内容不能为空"}), 400
101
102         logger.info(f"收到用户消息: {user_message}")
103
104         # 调用豆包API
105         ai_reply = DoubaoAIClient.generate_reply(user_message)
106
107         return jsonify({
108             "success": True,
109             "ai_reply": ai_reply,
110             "model": "doubao",
111             "api_key": app.config['DOUBAO_API_KEY'][:8] + "... " +
app.config['DOUBAO_API_KEY'][-4:],
112             "timestamp": datetime.now().isoformat()
113         })
114
115     except Exception as e:
116         logger.error(f"对话处理错误: {e}")
117         return jsonify({
118             "success": False,
119             "error": str(e),
120             "ai_reply": "管仲沉思中, 请稍候再试"

```

```
121         }, 500
122
123     # 豆包API健康检查
124     @app.route('/api/doubao/health', methods=['GET'])
125     def doubao_health():
126         """检查豆包API状态"""
127         try:
128             # 简单的测试请求
129             test_reply = DoubaoAIClient.generate_reply("你好")
130             return jsonify({
131                 "status": "healthy",
132                 "model": "doubao",
133                 "api_key_status": "有效",
134                 "response_time": "正常",
135                 "timestamp": datetime.now().isoformat()
136             })
137         except Exception as e:
138             return jsonify({
139                 "status": "unhealthy",
140                 "error": str(e),
141                 "api_key_status": "无效",
142                 "timestamp": datetime.now().isoformat()
143             }, 503)
144
145     # 静态文件服务
146     @app.route('/')
147     def serve_index():
148         """提供前端主页面"""
149         return send_from_directory('../frontend/public', 'guanzhong_ai.html')
150
151     @app.route('/<path:path>')
152     def serve_static(path):
153         """提供静态文件"""
154         return send_from_directory('../frontend/public', path)
155
156     # 应用启动
157     if __name__ == '__main__':
158         # 检查前端目录是否存在
159         frontend_path = os.path.abspath('../frontend/public')
160         if not os.path.exists(frontend_path):
161             logger.warning(f"前端目录不存在: {frontend_path}")
162
163         # 启动Flask应用
164         app.run(
165             debug=True,
166             port=5000,
167             host='0.0.0.0',
```

```
168         threaded=True
```

```
169     )
```